TP3 : Segmentation de maillage 3D

Géométrie Algorithmique - ISIMA F1

Nathan BERT
ISIMA - ZZ3
nathan.bert@etu.uca.fr

17 février 2026

Résumé

Ce rapport présente le travail réalisé dans le cadre du TP3 de géométrie algorithmique portant en partie sur la segmentation de maillages 3D. L'objectif du TP est de parcourir des maillages de points à l'aide de la librairie CGAL puis de segmenter ce maillage selon différents critères géométriques. Les résultats sont illustrés par des visualisations sous MeshLab sur différents modèles fournis. Par soucis de transparence quand à mon usage des outils de génération automatique, je tiens à préciser que le code produit a entièrement été commenté à l'aide de l'intelligence artificielle, en se basant sur le guide de style de la nasa ainsi que les affichages à l'execution. Le code et la logique implémentée sont cependant ma production.

1 Calcul de propriétés locales

Dans la première partir du TP nous nous sommes premièrement porté sur le calcul de différentes propriétés géométriques des maillages.

1.1 Genre du maillage

Le genre topologique d'un maillage fermé se calcule via la caractéristique d'Euler :

$$\chi = V - E + F = 2 - 2g$$

où V est le nombre de sommets, E le nombre d'arêtes, F le nombre de faces, et g le genre.

Pour le maillage `home-remeshed.off` :

- Sommets : 40 376
- Arêtes : 80 756
- Faces : 40 378
- Caractéristique d'Euler : $\chi = 40\,376 - 80\,756 + 40\,378 = -2$
- Genre : $g = (2 - (-2))/2 = 2$

On a donc que le maillage possède 2 trous (ici la porte et la fenetre de la maison).

1.2 Calcul d'aires (I.1)

Pour chaque face triangulaire, l'aire est calculée via la primitive CGAL :

$$A_{\text{triangle}} = \sqrt{\text{squared_area}(p_0, p_1, p_2)}$$

Seulement notre maillage peut comporter des faces rectangulaires, quisi elles sont mal prisent en compte ne seront pas évaluées à la bonne aire. Ainsi pour les faces d'un quadrilatérales de côtés (p_0, p_1, p_2, p_3), on calcule l'air des deux triangles formés par la diagonale (p_0, p_2) :

$$A_{\text{quad}} = A_{\triangle(p_0,p_1,p_2)} + A_{\triangle(p_2,p_3,p_0)}$$

Résultats :

- Aire totale du maillage : 137.62 unités^2
- Aire minimale d'une face : 0.0009
- Aire maximale d'une face : 0.0044

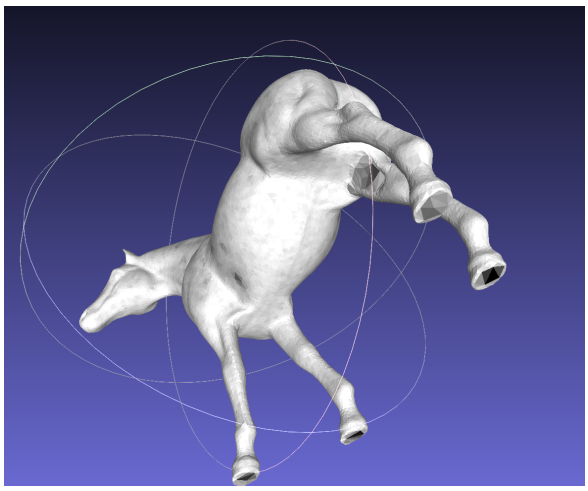
Les aires sont stockées dans une map qui nous sera utile ultérieurement, pour d'autres traitements.

1.3 Export coloré par aire (I.2)

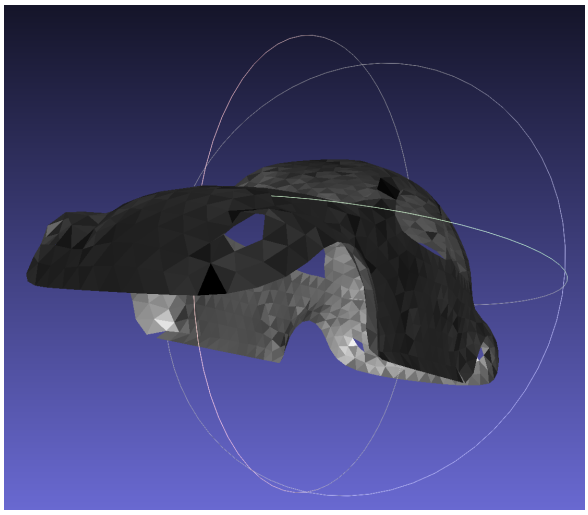
Les valeurs d'aire précédement trouvées sont normalisées dans l'intervalle $[0, 1]$ puis converties en niveaux de gris :

$$\text{gray} = \left\lfloor 255 \times \left(1 - \frac{A - A_{\min}}{A_{\max} - A_{\min}}\right)\right\rfloor$$

La figure 1 montre le maillage coloré : les faces claires ont une petite aire, les faces sombres ont une grande aire.



(a) Mesh de cheval



(b) Mesh coccinelle

FIGURE 1 – Mesh colorés selon la taille d'aire de leurs faces

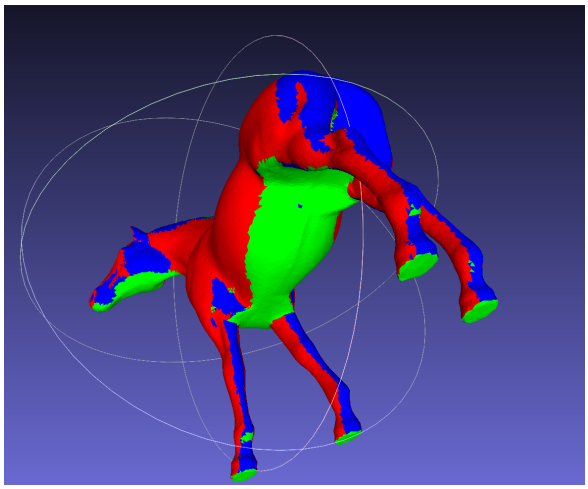
1.4 Angles des normales

Pour chaque face, on calcule le vecteur normal $\mathbf{n}$ puis on identifie la composante dominante :

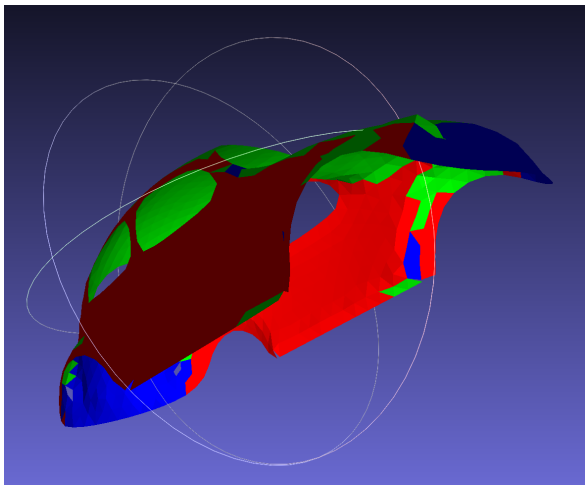
$$\text{axe dominant} = \arg \max_{i \in \{x,y,z\}} |\mathbf{n}_i|$$

Les faces sont colorées selon leur orientation :

- Rouge : normale proche de l'axe X (faces verticales est-ouest)
- Vert : normale proche de l'axe Y (faces verticales nord-sud)
- Bleu : normale proche de l'axe Z (faces horizontales)



(a) Mesh de cheval



(b) Mesh coccinelle

FIGURE 2 – Mesh colorés selon l'orientation de la normale de leurs faces

1.5 Distance euclidienne point-face

Pour integrer une seconde metrique calculée sur notre maillage j'ai implementé une fonction qui pour une face donnée et un point de référence, calcul la distance entre le point et le barycentre de la face.

Pour une face donnée, le barycentre G est calculé comme la moyenne arithmétique de ses sommets :

$$G = \frac{1}{n} \sum_{i=1}^{n} p_i$$

où n est le nombre de sommets de la face (3 pour un triangle, 4 pour un quad).

La distance euclidienne totale et ses composantes selon les axes sont calculées :

$$d_{\text{total}} = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$$

Cette fonction retourne un tableau de 4 valeurs : $[\Delta x, \Delta y, \Delta z, d_{\text{total}}]$, permettant des analyses par axe (distance horizontale $d_{xy} = \sqrt{\Delta x^2 + \Delta y^2}$, différence de hauteur $|\Delta z|$) ou la distance totale.

Cette fonction est utilisée dans la partie II pour déterminer les zones à portée du robot, en calculant la distance de chaque face au point de référence.

2 Segmentation par seuillage

Cette partie implémente une segmentation multi-tags basée sur des critères géométriques.

2.1 Tags géométriques (II.1)

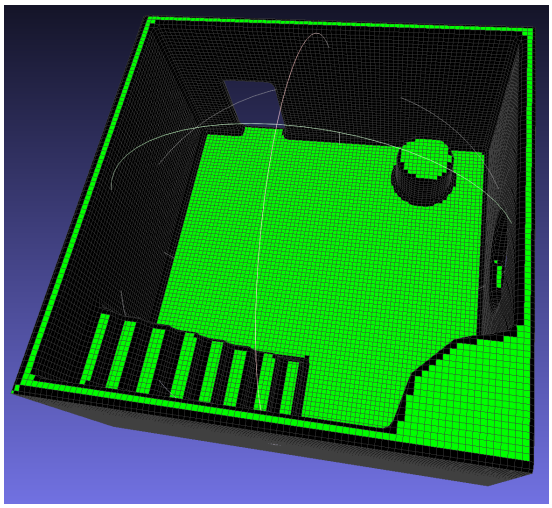
Chaque face du maillage se voit attribuer plusieurs tags booléens selon les critères suivants :

Tag	Critère	Couleur
`Grande`	$A > 5.0$	Rouge
`Plane`	$\cos(\theta) > 0.966$ (angle $< 15°$ avec Z)	Vert
`VerySteep`	$\cos(\theta) < 0.866$ (angle $> 30°$ avec Z)	Jaune
`OrienteHaut`	$\lvert\mathbf{n}_z\rvert > \max(\lvert\mathbf{n}_x\rvert, \lvert\mathbf{n}_y\rvert)$	Bleu

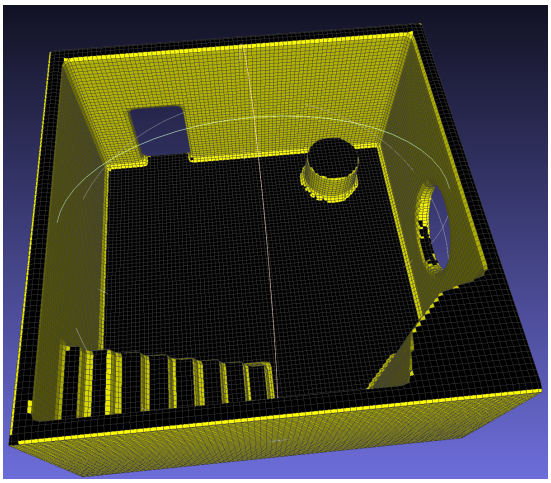
TABLE 1 – Tags géométriques et critères associés

où θ est l'angle entre la normale de la face et l'axe vertical Z.

Pour illustrer cette segmentation par Tag on peut visualiser les Tags qui nous interessent dans le cadre de notre partie II.



(a) Tag `Plane` (surfaces horizontales)



(b) Tag `VerySteep` (surfaces inclinées)

FIGURE 3 – Exemples de segmentation par tag géométrique

Ici on peut directement remarquer que le tag "Plane" permet un selection des zones sur lesquelles notre robot pourrait se déplacer en théorie s'il y était déposé, sans pour autant qu'il puisse y acceder depuis une autre zone plane.
Par exemple les marches, praticables individuellement mais notre robot ne circulant que sur les zones planes ne pourrait ni y entrer si il était disposé à l'exterieur si en sortir le cas contraire.

On comprend donc que pour determiner les zones accessibles à notre robot, il va nous falloir integrer d'autres critères discriminants sur les surfaces.

2.2 Scénario robotique (II.2)

On considère un scénario où un robot virtuel (de type mono-essieux) se déplace dans l'environnement modélisé. Trois classes de segmentation sont définies.

2.2.1 Zones de marche

Critère : Surfaces planes (angle $< 15°$ avec l'horizontal) sur lesquelles le robot peut circuler, ici la zone marchable est confondue avec le tag Plane, mais si d'autres critères étaient à prendre en compte sur les capacités du robot à se déplacer, alors elles pourraient ne pas etre confondues.

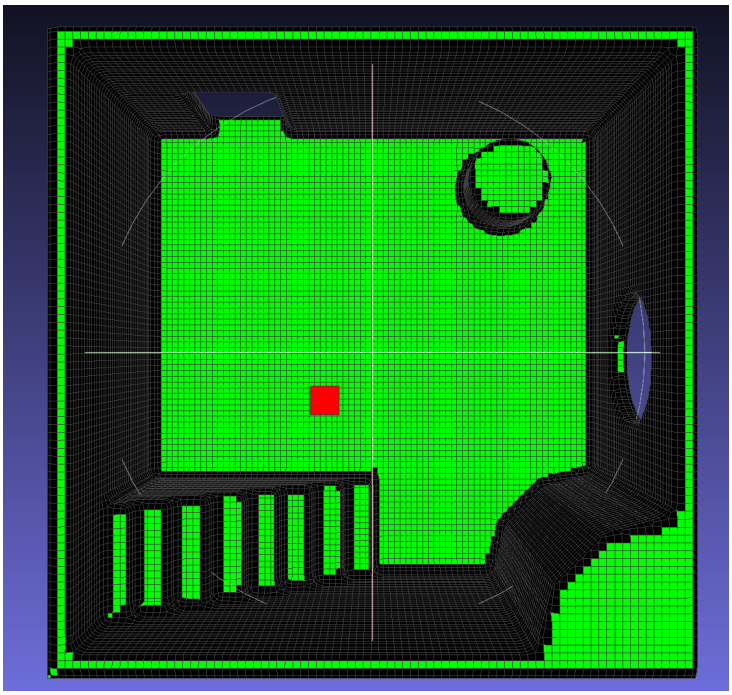


FIGURE 4 – Zones de marche (vert) : surfaces praticables pour le robot

La figure 4 montre que le sol de la pièce est correctement identifié comme zone marchable.

2.2.2 Obstacles

Critère : Surfaces inclinées (angle $> 30°$ avec l'horizontal) dont la distance au robot est inférieur à la distance de déplacement dans notre scénario (ici 2 mètres). Un critère de taille sur les faces obstacles est aussi définit mais fixé à 0 car peu utile dans ce contexte. Il faudrait egalement pouvoir determiner si un objet complet est un obstacle plutot qu'une seule face.

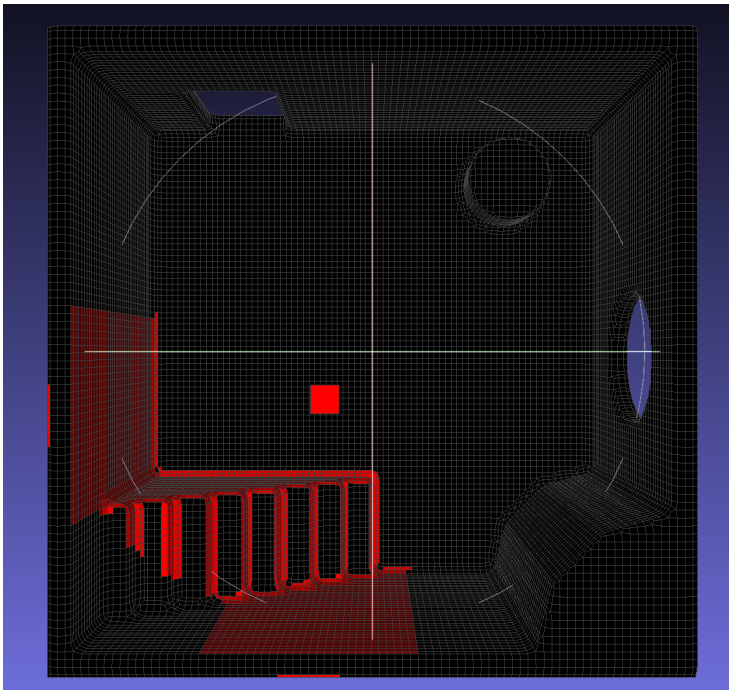


FIGURE 5 – Obstacles (rouge) : surfaces verticales bloquant le passage dans le rayon d'action du robot

Les murs, portes et fenêtres sont correctement détectés comme obstacles (figure 5).

2.2.3 Zones à portée

Critère : Surfaces planes proches du robot en distance horizontale ($d_{xy} < 2.0$ m) et en hauteur ($|\Delta z| < 0.2$ m).

La position du robot est fixée à $(x = 2, y = 2, z = 0)$. Pour chaque face, on calcule :

$$d_{xy} = \sqrt{(x_{\text{face}} - x_{\text{robot}})^2 + (y_{\text{face}} - y_{\text{robot}})^2}$$

$$\Delta z = z_{\text{face}} - z_{\text{robot}}$$

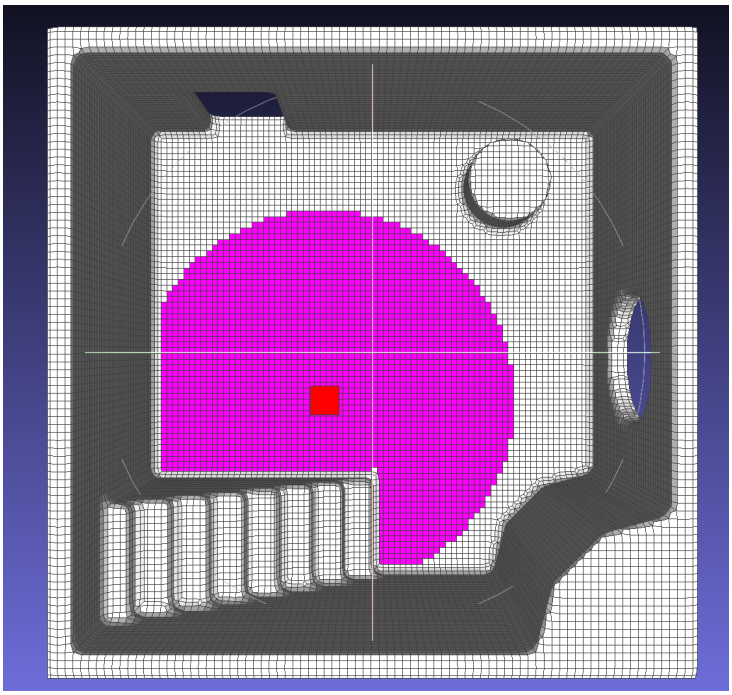


FIGURE 6 – Zones à portée (magenta) : surfaces accessibles depuis la position du robot

La figure 6 montre les zones d'interaction potentielles autour du robot (rayon de 2 mètres).

On constate que la zone qui est considérée comme à portée est à la fois la zone catégorisée comme praticable par le robot, respectant sa capacité de déplacement fixées par notre scénario mais aussi les limites de déplacement verticaux supposées de notre robot.

Ici on suppose un robot mono-essieux ne permettant de naviguer uniquement sur des zones à faible dénivellé et dont tout écart vertical de plus de 30cm par rapport à se position initiale, n'est pas possible. Pour cette hypothèse les marches, bien que praticables, se retrouvent excluent de la zone à portée du robot dans notre scénario.

3 Conclusion

En conclusion, ce TP m'aura permis de prendre en main la librairie de geometrie algorithmique CGAL pour réaliser l'extraction et l'étude de caractéritiques permettant nottament la segmentation d'un maillage en vu d'une exploitation plus poussée. Il m'aura nottament été permis d'implementer :

— Calcul de propriétés topologiques (genre) et locales (aire, orientation)

— Segmentation multi-critères par seuillage géométrique

Les résultats montrent qu'avec des critères simples, il est deja possible d'effectuer une segmentation sommaire de notre maillage en première approche, peu coûteuse en ressources, nous donnant déjà une vision plus concrète de notre probleme et de nos possibilités.

L'implémentation repose sur la bibliothèque CGAL pour les primitives géométriques (calcul d'aires, normales) et sur une structure de données efficace (`std::map`) pour le stockage des propriétés par face. Le code est organisé de manière modulaire, facilitant l'ajout de nouveaux tags ou critères de segmentation.

A Disposition du code

- `myAlgo.hpp/cpp` : Fonctions de calcul géométrique et segmentation
- `main.cpp` : Programme principal avec tests par partie du TP
- Toutes les sorties du programme se trouvent dans le dossier output

B Dépôt GitHub

Le code source complet est disponible sur le dépôt :
https://github.com/NathanBert/geom-algo-segmentation.git